

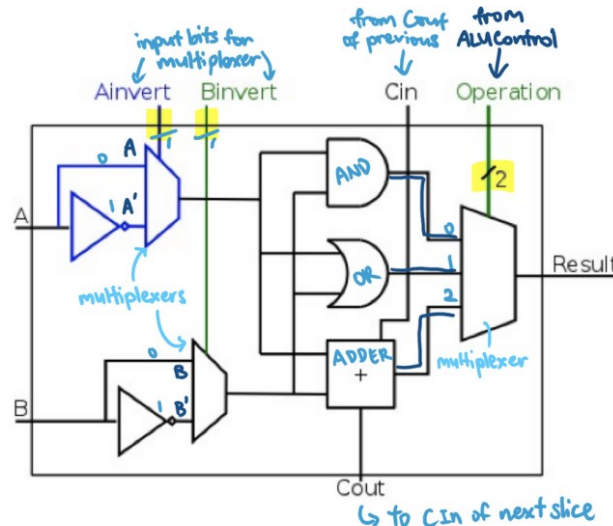
CS2100 Midterms Cheatsheet

12. Processor Control

Control Signals

- **RegDst** - (@Decode/OperandFetch) Selects the destination of WriteRegister in register file
 - 0/1: WriteRegister = Inst[20:16] (rt) / Inst[15:11] (rd / immediate)
- **RegWrite** - (@Decode/OperandFetch) Indicates if writing on WriteRegister is enabled
 - 0/1: No register write / WD write to WR
- **ALUSrc** - (@ALU) Determines the 2nd ALU input
 - 0/1: Read Data 2 / SignExtend(Inst15:0) (Sign extended Immediate)
- **ALUcontrol** - (@ALU) Determines the type of operation to execute
 - 0000: AND
 - 0001: OR
 - 0010: add
 - 0110: subtract
 - 0111: slt
 - 1100: NOR
- **MemRead** - (@Memory) Indicates if read from memory using Address returned by ALU
 - 0/1: No read / Reads memory with Address returned by ALU
- **MemWrite** - (@Memory) Indicates if write to memory using RD2 of Register File
 - 0/1: No memory write / Writes RD2 into the memory in Address returned by ALU
- **MemToReg** - (@RegWrite) Indicates which one to write back into WriteData in register file
 - 0/1: WriteData = ALU result / Memory read data
- **PCSrc** - (@ALU) Indicates whether to take branch address or not
 - 0/1: $PC = PC + 4$ / $PC = \text{SignExt}(\text{Inst}[15:0]) \ll 2 + (PC + 4)$
 - PCSrc = isBranch AND isZero

ALU



ALU operation controlled by 2-bit ALUControl

* Note: operation is determined by the last 2-bits *
Subtraction has Cin = 1 for bit-0

Multilevel Decoding

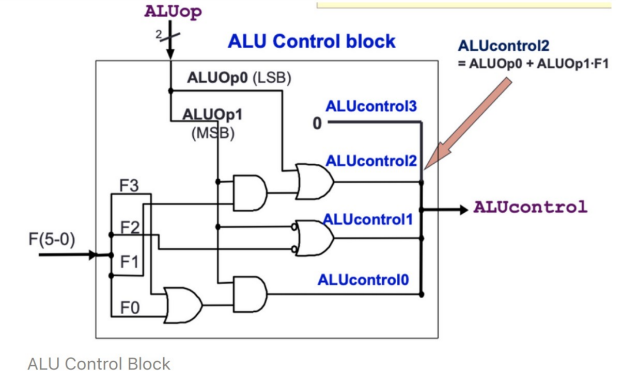
Used to generate the ALUcontrol signal. (Simpler and more efficient design than Brute Force approach)

1. Generate a 2-bit ALUOp signal from opcode
2. Use ALUOp and Function code (for R-type) to generate 4-bit ALUcontrol signal

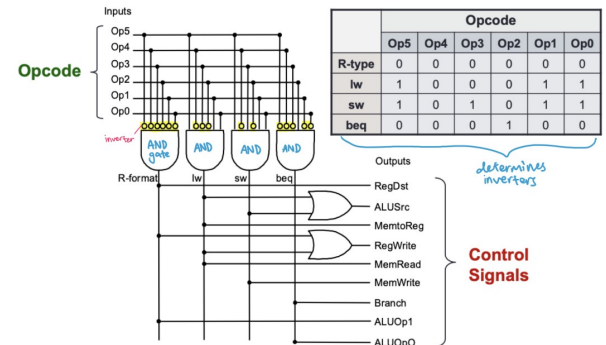
Opcode	ALUOp	Instruction Operation	Func field	ALU action	ALU control
lw	00	load word	x	add	0010
sw	00	store word	x	add	0010
beq	01	branch equal	x	subtract	0110
R-type	10	add	10 0000	add	0010
R-type	10	subtract	10 0010	subtract	0110
R-type	10	AND	10 0100	AND	0000
R-type	10	OR	10 0101	OR	0001
R-type	10	set on less than	10 1010	set on less than	0111

Generation of 2-bit ALUOp signal will be discussed later

ALUcontrol	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	slt
1100	NOR



Control Combinational Circuit



Instruction Execution

How the implementation of fetch, decode, memory, write, etc. are actually done

Mainly three things:

1. Read contents from register / memory
2. Perform some form of computation
3. Write results to register / memory

Single Cycle Execution

All performed within a clock period.

disadvantage: all instructions will take the same time as the slowest one.

Multi Cycle Execution

Break up instructions into execution steps (Fetch, Decode, ALU Operation, Mem read/write, Register Write). Each execution is one clock cycle. **disadvantage:** time taken depends on instructions and may not necessarily be faster.

Spaces and new lines

L^AT_EX ignores extra spaces and new lines. For example,

```
This sentence will look
fine after it is compiled.
```

This sentence will look fine after it is compiled.

Leave one full empty line between two paragraphs. Place `\\` at the end of a line to create a new line (but not create a new paragraph).

```
This
compiles
```

```
like\\
this.
This compiles
like
this.
Use \noindent to prevent a paragraph from indenting.
```

Comments

Use `%` to create a comment. Nothing on the line after the `%` will be typeset. `$f(x)=\sin(x)$ %this is the sine function` yields $f(x) = \sin(x)$

Delimiters

<i>description</i>	<i>command</i>	<i>output</i>
parentheses	<code>(x)</code>	(x)
brackets	<code>[x]</code>	$[x]$
curly braces	<code>\{x\}</code>	$\{x\}$

To make your delimiters large enough to fit the content, use them together with `\right` and `\left`. For example, `\left\{\sin\left(\frac{1}{n}\right)\right\}_n` produces

$$\left\{\sin\left(\frac{1}{n}\right)\right\}_n^\infty.$$

Curly braces are non-printing characters that are used to gather text that has more than one character. Observe the differences between the four expressions `x^2`, `x^{2}`, `x^2t`, `x^{2t}` when typeset: x^2 , x^2 , x^2t , x^{2t} .

Lists

You can produce ordered and unordered lists.

<i>description</i>	<i>command</i>	<i>output</i>
unordered list	<code>\begin{itemize}</code>	
	<code>\item</code>	
	Thing 1	• Thing 1
	<code>\item</code>	• Thing 2
	Thing 2	
ordered list	<code>\end{itemize}</code>	
	<code>\begin{enumerate}</code>	
	<code>\item</code>	
	Thing 1	1. Thing 1
	<code>\item</code>	2. Thing 2
	Thing 2	
	<code>\end{enumerate}</code>	

Symbols (in *math* mode)

The basics

<i>description</i>	<i>command</i>	<i>output</i>
addition	<code>+</code>	$+$
subtraction	<code>-</code>	$-$
plus or minus	<code>\pm</code>	$\pm$
multiplication (times)	<code>\times</code>	$\times$
multiplication (dot)	<code>\cdot</code>	$\cdot$
division symbol	<code>\div</code>	$\div$
division (slash)	<code>/</code>	$/$
circle plus	<code>\oplus</code>	$\oplus$
circle times	<code>\otimes</code>	$\otimes$
equal	<code>=</code>	$=$
not equal	<code>\neq</code>	$\neq$
less than	<code><</code>	$<$
greater than	<code>></code>	$>$
less than or equal to	<code>\leq</code>	$\leq$
greater than or equal to	<code>\geq</code>	$\geq$
approximately equal to	<code>\approx</code>	$\approx$
infinity	<code>\infty</code>	∞
dots	<code>1,2,3,\ldots</code>	$1, 2, 3, \dots$
dots	<code>1+2+3+\cdots</code>	$1 + 2 + 3 + \dots$
fraction	<code>\frac{a}{b}</code>	$\frac{a}{b}$
square root	<code>\sqrt{x}</code>	$\sqrt{x}$
<i>n</i> th root	<code>\sqrt[n]{x}</code>	$\sqrt[n]{x}$
exponentiation	<code>a^b</code>	a^b
subscript	<code>a_b</code>	a_b
absolute value	<code> x </code>	$ x $
natural log	<code>\ln(x)</code>	$\ln(x)$
logarithms	<code>\log_{a}b</code>	$\log_a b$
exponential function	<code>e^x=\exp(x)</code>	$e^x = \exp(x)$
degree	<code>\deg(f)</code>	$\deg(f)$

Functions

description	command	output
maps to	<code>\to</code>	$\rightarrow$
composition	<code>\circ</code>	$\circ$
piecewise function	<code>\begin{cases} x & x \geq 0 \\ -x & x < 0 \end{cases}</code>	$ x = \begin{cases} x & x \geq 0 \\ -x & x < 0 \end{cases}$

Greek and Hebrew letters

command	output	command	output
<code>\alpha</code>	α	<code>\tau</code>	τ
<code>\beta</code>	β	<code>\theta</code>	θ
<code>\chi</code>	χ	<code>\upsilon</code>	υ
<code>\delta</code>	δ	<code>\xi</code>	ξ
<code>\epsilon</code>	ϵ	<code>\zeta</code>	ζ
<code>\varepsilon</code>	ε	<code>\Delta</code>	Δ
<code>\eta</code>	η	<code>\Gamma</code>	Γ
<code>\gamma</code>	γ	<code>\Lambda</code>	Λ
<code>\iota</code>	ι	<code>\Omega</code>	Ω
<code>\kappa</code>	κ	<code>\Phi</code>	Φ
<code>\lambda</code>	λ	<code>\Pi</code>	Π
<code>\mu</code>	μ	<code>\Psi</code>	Ψ
<code>\nu</code>	ν	<code>\Sigma</code>	Σ
<code>\omega</code>	ω	<code>\Theta</code>	Θ
<code>\phi</code>	ϕ	<code>\Upsilon</code>	Υ
<code>\varphi</code>	φ	<code>\Xi</code>	Ξ
<code>\pi</code>	π	<code>\aleph</code>	$\aleph$
<code>\psi</code>	ψ	<code>\beth</code>	$\beth$
<code>\rho</code>	ρ	<code>\daleth</code>	$\daleth$
<code>\sigma</code>	σ	<code>\gimel</code>	$\gimel$

Set theory

description	command	output
set brackets	<code>\{1,2,3\}</code>	$\{1,2,3\}$
element of	<code>\in</code>	$\in$
not an element of	<code>\notin</code>	$\notin$
subset of	<code>\subset</code>	$\subset$
subset of	<code>\subseteq</code>	$\subseteq$
not a subset of	<code>\not\subset</code>	$\not\subset$
contains	<code>\supset</code>	$\supset$
contains	<code>\supseteq</code>	$\supseteq$
union	<code>\cup</code>	$\cup$
intersection	<code>\cap</code>	$\cap$
big union	<code>\bigcup_{n=1}^{10} A_n</code>	$\bigcup_{n=1}^{10} A_n$
big intersection	<code>\bigcap_{n=1}^{10} A_n</code>	$\bigcap_{n=1}^{10} A_n$
empty set	<code>\emptyset</code>	$\emptyset$
power set	<code>\mathcal{P}</code>	$\mathcal{P}$
minimum	<code>\min</code>	min
maximum	<code>\max</code>	max
supremum	<code>\sup</code>	sup
infimum	<code>\inf</code>	inf
limit superior	<code>\limsup</code>	lim sup
limit inferior	<code>\liminf</code>	lim inf
closure	<code>\overline{A}</code>	$\overline{A}$

Calculus

description	command	output
derivative	<code>\frac{df}{dx}</code>	$\frac{df}{dx}$
derivative	<code>f'</code>	f'
partial derivative	<code>\frac{\partial f}{\partial x}</code>	$\frac{\partial f}{\partial x}$
integral	<code>\int</code>	$\int$
double integral	<code>\iint</code>	$\iint$
triple integral	<code>\iiint</code>	$\iiint$
limits	<code>\lim_{x \rightarrow \infty}</code>	$\lim_{x \rightarrow \infty}$
summation	<code>\sum_{n=1}^{\infty} a_n</code>	$\sum_{n=1}^{\infty} a_n$
product	<code>\prod_{n=1}^{\infty} a_n</code>	$\prod_{n=1}^{\infty} a_n$

Logic

description	command	output
not	<code>\sim</code>	$\sim$
and	<code>\land</code>	$\wedge$
or	<code>\lor</code>	$\vee$
if...then	<code>\to</code>	$\rightarrow$
if and only if	<code>\leftrightarrow</code>	$\leftrightarrow$
logical equivalence	<code>\equiv</code>	$\equiv$
therefore	<code>\therefore</code>	$\therefore$
there exists	<code>\exists</code>	$\exists$
for all	<code>\forall</code>	$\forall$
implies	<code>\Rightarrow</code>	$\Rightarrow$
equivalent	<code>\Leftrightarrow</code>	$\Leftrightarrow$

Linear algebra

description	command	output
vector	<code>\vec{v}</code>	$\vec{v}$
vector	<code>\mathbf{v}</code>	$\mathbf{v}$
norm	<code> \vec{v} </code>	$ \vec{v} $
matrix	<code>\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}</code>	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$
determinant	<code>\det(A)</code>	$\det(A)$
determinant	<code>\operatorname{tr}(A)</code>	$\operatorname{tr}(A)$
dimension	<code>\dim(V)</code>	$\dim(V)$

Number theory

description	command	output
divides	<code> </code>	$ $
does not divide	<code>\not </code>	$\nmid$
div	<code>\operatorname{div}</code>	div
mod	<code>\mod</code>	mod
greatest common divisor	<code>\gcd</code>	gcd
ceiling	<code>\lceil x \rceil</code>	$\lceil x \rceil$
floor	<code>\lfloor x \rfloor</code>	$\lfloor x \rfloor$

Geometry and trigonometry

description	command	output
angle	<code>\angle ABC</code>	$\angle ABC$
degree	<code>90^\circ</code>	90°
triangle	<code>\triangle ABC</code>	$\triangle ABC$
segment	<code>\overline{AB}</code>	$\overline{AB}$
sine	<code>\sin</code>	sin
cosine	<code>\cos</code>	cos
tangent	<code>\tan</code>	tan
cotangent	<code>\cot</code>	cot
secant	<code>\sec</code>	sec
cosecant	<code>\csc</code>	csc
inverse sine	<code>\arcsin</code>	arcsin
inverse cosine	<code>\arccos</code>	arccos
inverse tangent	<code>\arctan</code>	arctan

Symbols (in *text* mode)

The followign symbols do **not** have to be surrounded by dollar signs.

description	command	output
dollar sign	<code>\\$</code>	$\$$
percent	<code>\%</code>	$\%$
ampersand	<code>\&</code>	$\&$
pound	<code>\#</code>	$\#$
backslash	<code>\textbackslash</code>	$\backslash$
left quote marks	<code>‘ ‘</code>	‘ ‘
right quote marks	<code>’ ’</code>	’ ’
single left quote	<code>‘</code>	‘
single right quote	<code>’</code>	’
hyphen	<code>X-ray</code>	X-ray
en-dash	<code>pp. 5--15</code>	pp. 5–15
em-dash	<code>Yes---or no?</code>	Yes—or no?

Resources

Great symbol look-up site: [Detexify](#)
 $\LaTeX$ Mathematical Symbols
The Comprehensive $\LaTeX$ Symbol List
The Not So Short Introduction to $\LaTeX$ 2 ϵ
TUG: The $\TeX$ Users Group
CTAN: The Comprehensive $\TeX$ Archive Network

$\LaTeX$ for the Mac: [Mac \$\TeX\$](#)
 $\LaTeX$ for the PC: [\$\TeX\$ NicCenter](#) and [MiK \$\TeX\$](#)
 $\LaTeX$ online: [Write \$\LaTeX\$](#) .